Session Fixation and Session Hijacking

Introduction

Web applications rely on session management mechanisms to maintain user authentication and state across stateless HTTP requests. Weaknesses in these mechanisms expose applications to serious security threats, notably session fixation and session hijacking attacks. These attacks allow adversaries to impersonate authenticated users without obtaining their credentials, leading to unauthorized access, data breaches, and privilege escalation. This report examines the concepts, attack vectors, threat models, and impacts of session fixation and session hijacking, and evaluates effective mitigation strategies for securing web sessions.

Purpose

Although session fixation and session hijacking belong to different classes of attacks, they serve a common purpose of gaining access to the victim's confidential information.

Differences

Session Fixation - ***aims to control or predict the session identifier before authentication***. By forcing a victim to log in using a session ID known to the attacker, the attacker can later reuse that same session to impersonate the user.

Session Hijacking - ***aims to steal an active session identifier after authentication***. Once the session token is obtained, the attacker can assume the victim’s identity and access protected resources.

Methodologies and Approach

- Conceptual and analytical research methodology

  This is done to examine session fixation and hijacking vulnerabilities in web applications. In this approach we combine theoretical analysis with practical evaluation to understand attack mechanisms and assess effective mitigation strategies.

- Identify common causes

  This could be due to improper handling of session IDs by the developers or human errors by the users.

- Analyzing real-world attack scenarios

  Attackers reuse known or stolen session IDs to impersonate authenticated users.

- Examine vulnerable session handling practices

  Attackers reuse known or stolen session IDs to impersonate authenticated users.

- Compare insecure implementations with secure best practices

  Insecure systems reuse session IDs, while secure systems regenerate and protect them.

- Evaluate mitigation techniques and their effectiveness

  Session rotation and secure cookies effectively prevent unauthorized session reuse.

Types of Session Hijacking

Session hijacking has two types, active and passive.

Active - Taking over when the user is actually using the service.

Passive - Secretly monitor the communication without interfering with the user-service interaction.

Techniques

There are several techniques which can be used to perform session hijacking

Bruteforcing

In this method attackers repeatedly guess session identifiers until a valid session is found, which is possible when session IDs are short, predictable, or poorly randomized.

XSS

XSS stands for Cross-site scripting which allows attackers to execute malicious scripts in a victim's browser to read and exfiltrate session cookies or tokens.

Session ID Prediction

Weak session generation algorithms enable attackers to predict future or existing session IDs based on patterns or insufficient entropy.

Packet Sniffing

On unencrypted connections, attackers can capture network traffic to extract session identifiers transmitted in plaintext.

Cookie Theft

Session cookies can be stolen through XSS, Malware, or insecure storage when proper cookie flags are not enforced.

Session Token Theft

Attackers obtain authentication tokens from client-side storage or exposed APIs and reuse them to impersonate legitimate users.

Mitigations

Session hijacking or fixation can be avoided by

Regenerating session ID upon authentication

Always issue a new session identifier after successful login to prevent attackers from reusing pre-authenticated sessions.

Enforcing single application session per user

Bind each authenticated session to one user account and invalidate any overlapping or duplicate sessions when required.

Separating unauthenticated and authenticated sessions

Use distinct session contexts so that unauthenticated sessions are never upgraded into authenticated ones.

Invalidating old sessions on privilege change

Destroy existing sessions when user roles or permissions change.

Using secure session attributes

Apply Secure, HttpOnly, and SameSite flags to all session cookies.

Limiting session lifetime

Enforce short expiration times and idle timeouts to reduce exposure.

Enforcing HTTPS for all traffic

Prevent session leakage through network interception.

Demonstration 1

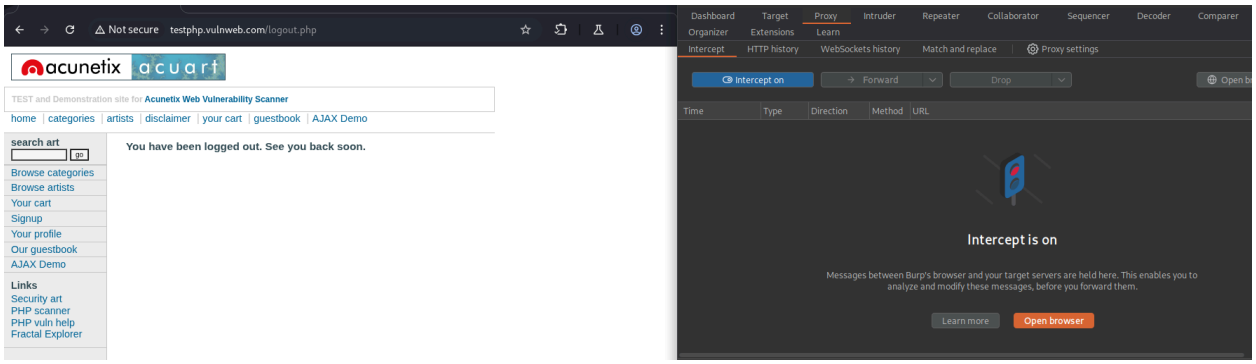
In this demonstration of session hijacking we try to gain information about how a site does its session management. It is typically unique for each site.

We use a popular site - http://testphp.vulnweb.com

This site is intentionally made vulnerable for session hijacking and fixation attacks.

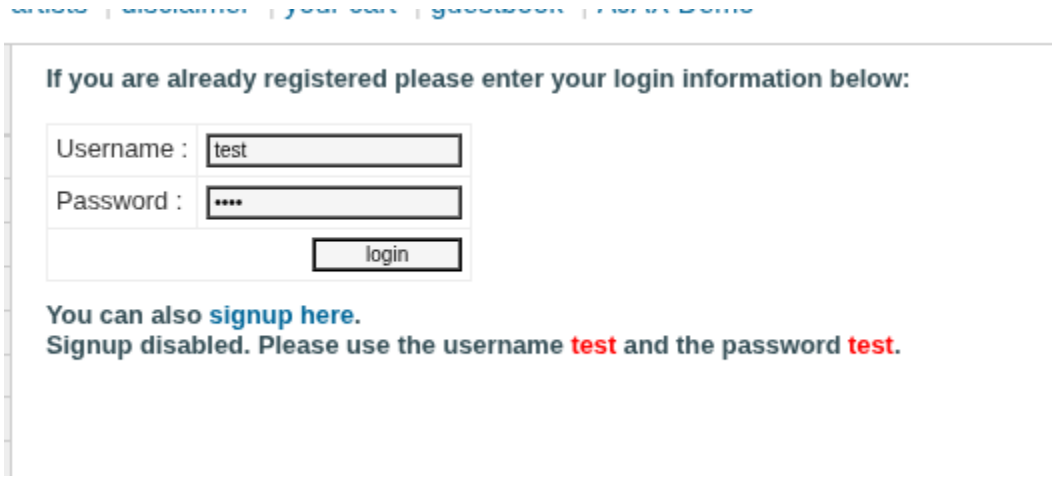
Tools used: BurpSuite

First, we open burpsuite and within the burpsuite browser we open the target site.

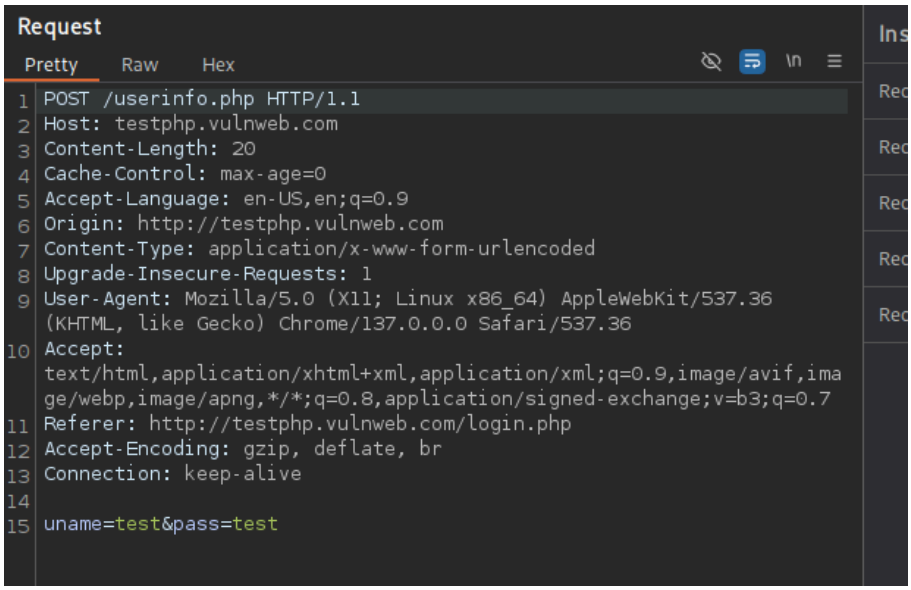


We then proceed to login via test credentials. That being

Username: test
Password: test

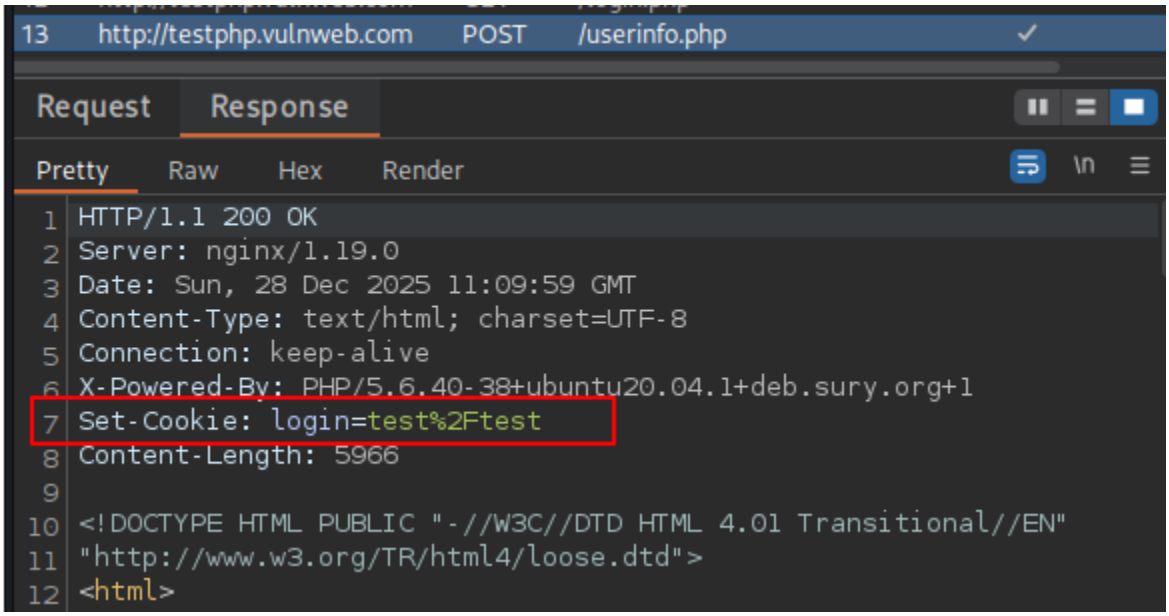


During the login process, we use burpsuite to intercept the request and see the request that is sent.



Here we see that the site makes a post request over insecure http sending uname (username) and pass (password) in plain text.

Once the login is complete, the site forwards us to /userinfo.php
With a response that contains cookies that are used for session management.



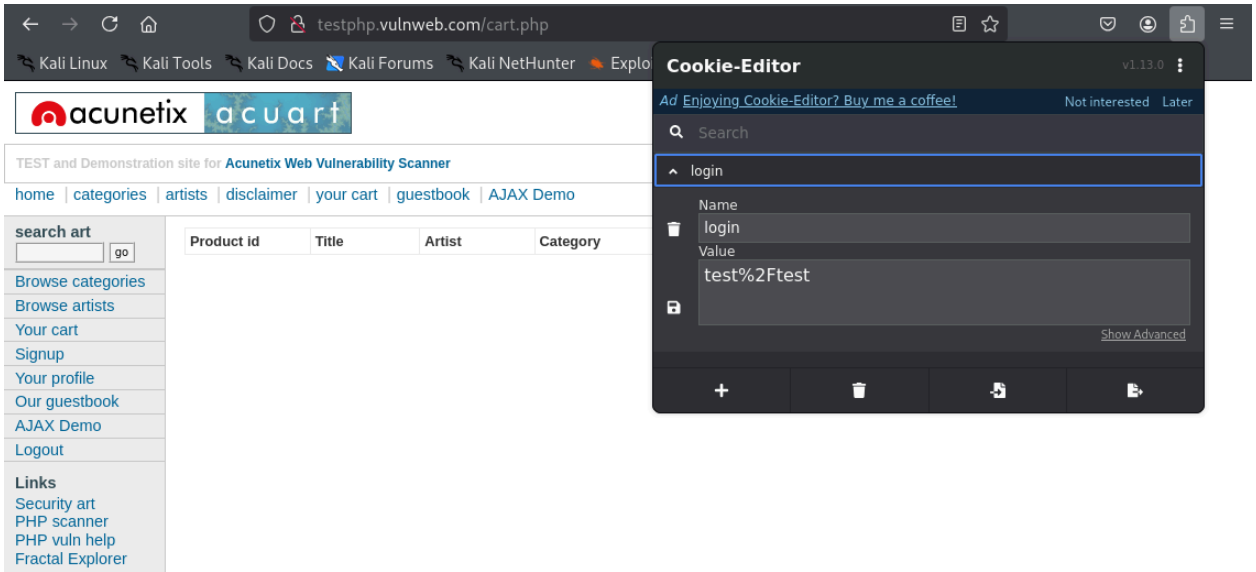
Demonstration 2

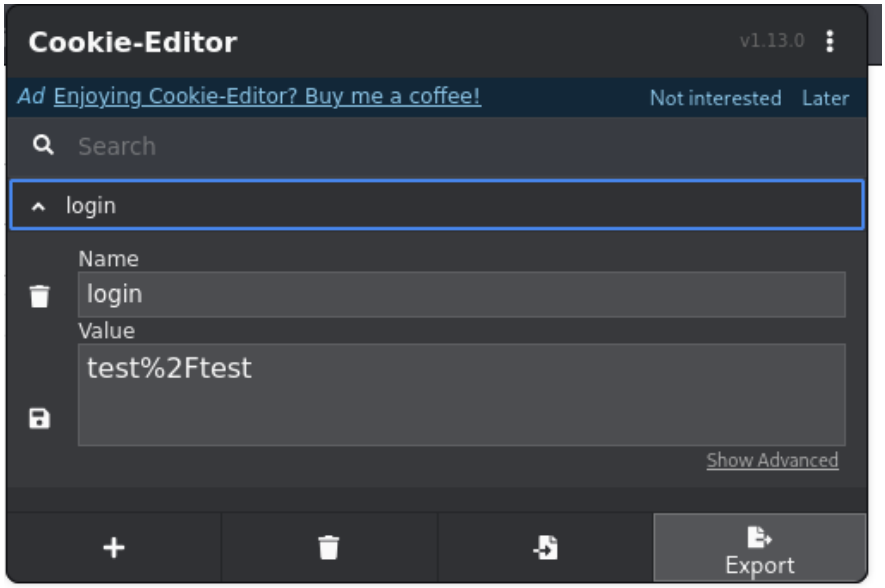
In this demonstration we try to hijack potentially any website by stealing the cookies and transferring them to a different browser on a different computer.

First, we need to install an extension called “cookie editor” onto both the victim's browser and ours.

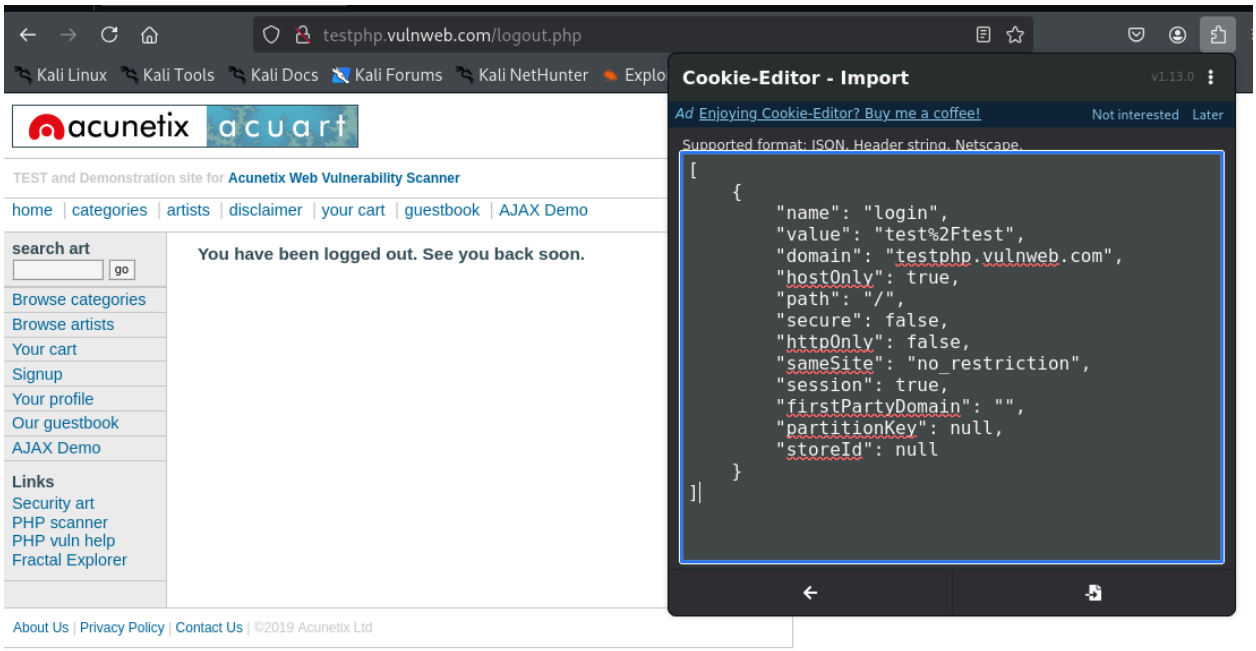
Then, we open the target site on our victim’s browser where the user is already logged in.

Use the cookie editor extension to export all the cookies as a json string.





This exported content can then be used to import onto a different browser on a different pc, effectively giving us full access to the user's session, albeit for a temporary period of time.



Here we import the cookie onto a different browser.

Conclusion

Session fixation and session hijacking represent critical threats stemming from insufficient web session management. Session fixation exploits the predictable nature or forced use of a known session ID *before* authentication, allowing an attacker to upgrade a passive session into an active, authenticated one once the victim logs in. In contrast, session hijacking involves the *theft* of an active, authenticated session token (often a cookie) to impersonate the legitimate user. Both attacks bypass traditional password authentication and lead directly to unauthorized access.

Effective mitigation hinges on robust session handling practices. The most crucial defence against session fixation is the **regeneration of the session ID upon successful authentication**, preventing the reuse of any pre-login identifier. For session hijacking, defenses include mandatory **HTTPS enforcement** to prevent token sniffing, applying **Secure** and **HttpOnly** flags to session cookies to counter XSS and network leakage, implementing strict **session timeouts**, and binding sessions to specific user context (like IP address). The demonstrations clearly illustrated how lax security practices, such as transmitting credentials over HTTP and failing to protect session cookies, enable these attacks. By adopting these security best practices, developers can significantly reduce the attack surface and protect user sessions against unauthorized takeover.